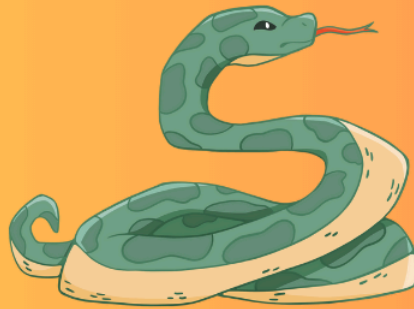


Creating a Snake Game using C++ (With Source Code)

Snake Game using C++



Complete Source Code

we will learn how to create a Creating a Snake Game using C++ (With Source Code).

The Snake game is a basic game implemented in C++ programming language with the logic using a ground matrix and provides a console-based game interface for the player to control the snake and eat food items while avoiding collisions.

Snake is one of the most popular games accessible on all sorts of devices and operating systems. In this game, snakes may travel in any direction, such as left, right, up, and down; after eating, the snake's length grows. Food for the Snake will be created at regular intervals.

The code implements the Snake game's essential logic, such as snake movement, collision detection, food creation, and game over situations.

Source code :

You can directly use this code by copying it in your IDE to understand the working and then can create it by understanding the project.

```
#include <iostream>
```

```
#include <time.h>
```

```
#include <cstdlib>
```

```
#include <windows.h>
```

```
#include <process.h>
```

```
#include <conio.h>
```

```
using namespace std;
```

```
#define MAX 100
```

```
#define WIDTH 77
```

```
#define HEIGHT 22
```

```
#define INIT_SNAKE_LENGTH 4
```

```
#define FOOD 1
```

```
//negative values represent areas that cannot be touched or  
else game over
```

```
#define WALL -2
```

```
#define SNAKE -1
```

```
#define NOTHING 0
```

```
#define RIGHT 0
```

```
#define UP 1
```

```
#define LEFT 2
```

```
#define DOWN 3
```

```
#define EXIT -1
```

```
static int dx[5] = { 1, 0, -1, 0 };
```

```
static int dy[5] = { 0, -1, 0, 1 };
```

```
//direction array, for use with RIGHT, UP, LEFT, DOWN  
constants
```

```
 //(1, 0) is RIGHT
```

//(0, -1) is UP (because the row number increase from top to bottom)

//(-1, 0) is LEFT

//(0, 1) is DOWN

```
int input = RIGHT; //global variable to take in the user's input
```

```
int item = NOTHING;
```

```
void gotoxy(int column, int row)
```

```
{
```

```
    HANDLE hStdOut;
```

```
    COORD coord;
```

```
    hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);
```

```
    if (hStdOut == INVALID_HANDLE_VALUE) return;
```

```
    coord.X = column;
```

```
    coord.Y = row;
```

```
    SetConsoleCursorPosition(hStdOut, coord);
```

```
}
```

```
void clearScreen()
```

```
{
```

```
    HANDLE                hStdOut;
```

```
    CONSOLE_SCREEN_BUFFER_INFO csbi;
```

```
    DWORD                  count;
```

```
    DWORD                  cellCount;
```

```
    COORD                  homeCoords = { 0, 0 };
```

```
    hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);
```

```
    if (hStdOut == INVALID_HANDLE_VALUE) return;
```

```
    /* Get the number of cells and cell attributes in the current  
buffer */
```

```
    if (!GetConsoleScreenBufferInfo(hStdOut, &csbi)) return;
```

```
    cellCount = csbi.dwSize.X *csbi.dwSize.Y;
```

```
    /* Fill the entire buffer with spaces */
```

```
if (!FillConsoleOutputCharacter(  
    hStdOut,      //handle to console screen buffer  
    (TCHAR) ' ', //character to write to the buffer  
    cellCount,    //number of cells to write to  
    homeCoords,   //coordinates of first cell  
    &count        //receives the number of characters written  
)) return;
```

```
/* Fill the entire buffer with the current colors and attributes  
*/
```

```
if (!FillConsoleOutputAttribute(  
    hStdOut,      //handle to console screen buffer  
    csbi.wAttributes, //Character attributes to use  
    cellCount,    //Number of cells to set attribute  
    homeCoords,   //Coordinate of first cell  
    &count        //receives the number of characters  
written  
)) return;
```

```
/* Move the cursor home */
```

```
    SetConsoleCursorPosition(hStdOut, homeCoords);  
}
```

//check if 2 directions are opposite

```
int oppositeDirection(int input1, int input2)  
{  
    if (input1 == LEFT && input2 == RIGHT)  
        return 1;  
    if (input1 == RIGHT && input2 == LEFT)  
        return 1;  
    if (input1 == UP && input2 == DOWN)  
        return 1;  
    if (input1 == DOWN && input2 == UP)  
        return 1;  
  
    return 0;  
}
```

```
struct Coordinate
```

```
{  
    int x, y;  
};
```

class snake

```
{  
private:  
    int length;  
    Coordinate body[WIDTH*HEIGHT];  
    int direction;  
    int ground[MAX][MAX];  
    int foodCounter;  
public:  
    void initGround();  
    void initSnake();  
    void updateSnake(int delay);  
    void updateFood();  
    void firstDraw();  
    int getFoodCounter();
```

```
};
```

```
void snake::initGround()
{
    int i, j;
    for (i = 0; i < MAX; i++)
        for (j = 0; j < MAX; j++)
            ground[i][j] = 0;

    for (i = 0; i <= WIDTH + 1; i++)
    {
        //top bottom wall
        ground[0][i] = WALL;
        ground[HEIGHT + 1][i] = WALL;
    }
    for (i = 0; i <= HEIGHT + 1; i++)
    {
        //right left wall
        ground[i][0] = WALL;
```

```
        ground[i][WIDTH + 1] = WALL;
    }
}

void snake::initSnake()
{
    length = INIT_SNAKE_LENGTH;    //set head of snake to be
    at the centre

    body[0].x = WIDTH / 2;
    body[0].y = HEIGHT / 2;
    direction = input;
    foodCounter = 0;

    int i;
    for (i = 1; i < length; i++)
    {
        body[i].x = body[i - 1].x - dx[direction];    //if moving
right,
        body[i].y = body[i - 1].y - dy[direction];    //body is on the
left
```

```
}  
  
//let the ground know the snake's position  
for (i = 0; i < length; i++)  
    ground[body[i].y][body[i].x] = SNAKE;  
}
```

```
void snake::updateSnake(int delay)  
{  
    int i;  
    Coordinate prev[WIDTH*HEIGHT];  
    for (i = 0; i < length; i++)  
    {  
        prev[i].x = body[i].x;  
        prev[i].y = body[i].y;  
    }
```

```
    if (input != EXIT && !oppositeDirection(direction, input))  
        direction = input;
```

```
body[0].x = prev[0].x + dx[direction];    //head of snake  
body[0].y = prev[0].y + dy[direction];    //follows the  
direction
```

```
if (ground[body[0].y][body[0].x] < NOTHING)  
{  
    item = -1;  
    return;  
}
```

```
if (ground[body[0].y][body[0].x] == FOOD)  
{  
    length++;    //length of snake increases when it eats  
food
```

```
    item = FOOD;  
}  
else  
{  
    ground[body[length - 1].y][body[length - 1].x] =  
NOTHING;
```

```

    item = NOTHING;

    gotoxy(body[length - 1].x, body[length - 1].y);    // if
snake does not get food,

    cout << " ";    // erase last part because the
snake is moving

}

for (i = 1; i < length; i++)

{
    body[i].x = prev[i - 1].x;    //body follows the previous
    body[i].y = prev[i - 1].y;    //location that it was from
}

gotoxy(body[1].x, body[1].y);

cout << "+";    //change the previous head to a body
gotoxy(body[0].x, body[0].y);

cout << "O";    //add a head to the snake

//let the ground know the snake's position
for (i = 0; i < length; i++)

```

```
ground[body[i].y][body[i].x] = SNAKE;
```

```
Sleep(delay);
```

```
return;
```

```
}
```

```
void snake::updateFood()
```

```
{
```

```
int x, y;
```

```
do
```

```
{
```

```
    x = rand() % WIDTH + 1;
```

```
    y = rand() % HEIGHT + 1;
```

```
} while (ground[y][x] != NOTHING);
```

```
ground[y][x] = FOOD;
```

```
foodCounter++;
```

```
gotoxy(x, y);
```



```

    || (i == HEIGHT + 1 && j == WIDTH + 1))

    cout << "+"; //the 4 corners

else

    if (j == 0 || j == WIDTH + 1)

        cout << "|"; //left/right wall

    else

        cout << "-"; //top/bottom wall

    break;

case SNAKE:

    if (i == body[0].y && j == body[0].x)

        cout << "O";

    else

        cout << "+";

    break;

default: //food

    cout << "\u2022"; //prints a middle dot

}

}

cout << endl;

```

```
}  
}
```

```
int snake::getFoodCounter()  
{  
    return foodCounter;  
}
```

```
void userInput(void* id)  
{  
    do  
    {  
        int c = _getch();  
        switch (c)  
        {  
            case 'W': case 'w': input = UP ; break;  
            case 'S': case 's': input = DOWN ; break;  
            case 'D': case 'd': input = RIGHT; break;  
            case 'A': case 'a': input = LEFT ; break;
```

```
        case 27:      input = EXIT ; break;
    }
} while (input != EXIT && item >= 0);

_endthread();

return;
}

int main()
{
    int delay = 50;
    srand(time(NULL));
    snake nagini;
    nagini.initGround();
    nagini.initSnake();
    nagini.updateFood();
    nagini.firstDraw();
    _beginthread(userInput, 0, (void*)0);
```

```
do
{
    nagini.updateSnake(delay);
    if (item == FOOD)
        nagini.updateFood();
} while (item >= 0 && input != EXIT);

gotoxy(WIDTH / 2 - 5, HEIGHT / 2 - 2);
cout << "GAME OVER";
gotoxy(WIDTH / 2 - 8, HEIGHT / 2 + 2);
cout << "Your score is " << nagini.getFoodCounter() - 1 << "!"
<< endl;

gotoxy(WIDTH / 2, HEIGHT / 2);

_getch();
return 0;
}
```

OUTPUT :-





Now let us understand the code:-

- We will start by writing the header of the code with the required libraries – `<iostream>` (input and output stream), `<time.h>`: Contains functions for manipulating time, `<cstdlib>`: Includes functions for generating random number, `<windows.h>`: Allows interaction with the Windows API for console handling, `<process.h>`: Provides functions for creating threads, `<conio.h>`: Enables console input/output functions.
- Now we will define some constants – MAX: This parameter specifies the maximum size of the ground matrix.
WIDTH and HEIGHT: Define the gaming area's width and

height.

INIT_SNAKE_LENGTH: Specifies the snake's starting length.

FOOD: The monetary worth of a food item.

WALL, SNAKE, NOTHING: These values represent walls, snake bodies, and empty spaces, respectively.

RIGHT, UP, LEFT, DOWN: These are the snake movement directions.

EXIT: Displays the exit code.

- Defining the global variables – dx and dy: Arrays representing the x and y coordinate changes in each direction.

input: Holds the user's input direction.

item: Represents the current item on the ground (food, nothing, or game over).

- We will create the functions:
- The gotoxy() function will moves the cursor to the specified position on the console screen. It uses an if statement.
- The clearScreen() function will clear the console. The code functionality is specified in the source code as comments for the detailed explanation.
- The oppositeDirection() function will check if two directions are opposite.
- We will create a structure. The Coordinate structure will hold 2 variables – x and y to store coordinate values.
- Now creating a class called snake to hole information related the snake in the game. It will contain private data

members –

- int length;
- Coordinate body[WIDTH*HEIGHT];
- int direction;
- int ground[MAX][MAX];
- int foodCounter
- The public member functions –
- void initGround();
- void initSnake();
- void updateSnake(int delay);
- void updateFood();
- void firstDraw();
- int getFoodCounter();
- We will define the functions of the class outside the class using scope resolution operator.
- The initGround() function will initialize the ground matrix with walls.
- initSnake: Initializes the snake's position, length, and direction.
- updateSnake: Updates the snake's movement and handles collisions.
- updateFood: Updates the position of the food item.
- firstDraw: Draws the initial game screen.
- getFoodCounter: Returns the number of food items eaten by the snake.
- The main() function will initialize the variables and objects that are required. This method creates a thread to process user input. The game loop is executed, updating the

snake's movement and checking for game over situations. The game over screen and the player's score are displayed.